Université Grenoble Alpes
L3 Informatique

Année 2023-2024

ALM - Quick 18/10/2023

Durée = 1h
Aucun document autorisé.

NOTE IMPORTANTE. *Pour toutes les questions où il est demandé d'expliquer la réponse, une réponse sans explication sera notée 0.*

NOM : ……..……..……..…….. **PRENOM** : ……..

Exercice 1. *14 points*

Le problème considéré ici concerne le *contrôle de feux tricolores à l'intersection d'une route principale et d'une route secondaire* très peu fréquentée.
Pour fluidifier la circulation, le principe consiste à laisser le feu au vert sur la route principale le plus longtemps possible (et donc le feu au rouge sur la route secondaire). Les feux ne s'inverseront que lorsqu'un nombre minimum de voitures seront arrivées au feu rouge sur la route secondaire, nous considérerons 12 voitures ici. Son feu restera alors au vert (et celui de la route principale au rouge) pour un temps constant (qui a préalablement été estimé suffisant pour que la quantité de voitures généralement présentes aient le temps de franchir le feu).
NOTE. Pour simplifier le problème, nous ne considérerons pas le passage par le feu orange.

Notre objectif serait d'utiliser la carte STM32F4 pour **piloter le feu sur la route principale**.
Nous allons ici seulement simuler le passage au feu vert ou rouge par l'allumage de la *led* correspondante. L'évolution de l'allumage des leds se fera à la fréquence du *systick*.
Afin de simuler également le comptage des voitures se présentant au feu rouge sur la route secondaire, nous utiliserons le *générateur de nombres aléatoires*.

Quelques rappels concernant ces périphériques, et des macros à utiliser dans les questions de cet exercice, sont fournis en Annexe.

Le code développé ci-dessous utilisera de plus les définitions/déclarations suivantes :

```
// Nombre de voitures au-delà duquel le feu sur la route principale doit
// passer au rouge :
#define MAX_CAR 12
// Nombre de périodes du systick pendant feu rouge sur route principale :
#define LIMIT_RED 6

uint32_t rnd = 0;   // valeur récupérée du RNG
unsigned int nb_car = 0;      // nombre de voitures sur route secondaire
int green = 1;     // 1 si feu vert, 0 si feu rouge sur route principale
unsigned int count_time_red = 0;   // comptage des périodes au feu rouge
```

Q1. En utilisant les définitions fournies, définir une **macro** `GPIO_OUT` qui permet d'accéder au contenu du registre de sortie du port GPIO auquel sont connectées les leds, comme un entier non signé 32 bits (on rappelle que le type correspondant est `uint32_t`).

Réponse. 1 pt

Cf cours 1 (slides 43 à 47 + vos notes de cours) et exercice 2 du TD2 semaine 1

Q2. Expliquer selon quelle **méthode** (i.e., en modifiant quoi, comment, pourquoi ?) vous pourrez procéder à un **allumage** des leds qui sera caractérisé par une **valeur `val`** : `val` est un entier compris entre 0 et 15, chaque bit à 1 de ce code sur 4 bits indique une led à allumer.

Réponse. 1 pt

Cf TD/TP semaine 3 (et repris encore dans les semaines suivantes), en particulier tous les détails expliqués pas à pas dans l'exercice 2 du TD.

Q3. Recopier et compléter le **code de la fonction** *allume_leds* ci-dessous selon ce que vous avez conçu aux questions 2 et 1.

```
void allume_leds (uint16_t val) {
    // compléter
}
```

Réponse. 1 pt

Idem, cf TD/TP semaine 3 (et repris encore dans les semaines suivantes), en particulier tous les détails expliqués pas à pas dans l'exercice 2 du TD.

Comme indiqué précédemment, nous allons maintenant simuler le passage au feu vert ou rouge sur la route principale par **l'allumage de la led** correspondante.
Afin de simuler le **comptage** des voitures se présentant au feu rouge sur la route secondaire, nous utiliserons le **générateur de nombres aléatoires** en mode interruption (il est supposé avoir été configuré en conséquence).

Q4. Expliquer comment s'appellera (pourquoi) et tout ce que fera techniquement la fonction qui sera chargée de **récupérer dans la variable globale `rnd`** *un nouveau nombre aléatoire généré*, automatiquement chaque fois qu'il est disponible (ne pas oublier qu'il faut vérifier qu'un nouveau nombre aléatoire est bien prêt, cf Annexe).

En utilisant les définitions fournies, **écrire** cette fonction.

Préciser ce qui **provoquera l'appel** de cette fonction lorsqu'un nouveau nombre est disponible.

Réponse. 2,5 pts

Cf TP2 semaine 4 (version simplifiée ici, sans prise en compte des erreurs)

Q5. L'objectif maintenant est **d'adapter l'allumage des leds** aux circonstances, à la fréquence du *systick* (il est supposé avoir été configuré en conséquence). L'adaptation de l'allumage se fera comme suit :

- lorsque le feu sur la route principale est au vert, le nombre de voitures présentes sur la route secondaire `nb_car` est incrémenté d'une valeur comprise entre 0 et 2, calculée en utilisant un nombre généré aléatoirement.
Si `nb_car` dépasse le nombre de voitures `MAX_CAR` au-delà duquel le feu sur la route secondaire doit passer au vert, alors le feu sur la route principale est mis au rouge (et il devra y rester pendant un nombre de périodes du systick donné par `LIMIT_RED`)

- lorsque le feu sur la route principale est au rouge, si le temps pendant lequel il devait rester rouge est écoulé (limite `LIMIT_RED` atteinte pour la variable `count_time_red`), alors il est mis au vert, et le nombre de voitures comptées sur la route secondaire est réinitialisé (i.e., on suppose que toutes les voitures ont eu le temps de franchir le feu).

Ecrire la **fonction** qui permettra de réaliser automatiquement ce comportement. **Expliquer** soigneusement le nom et le code de cette fonction (par exemple en le commentant avec soin).

Préciser ce qui **provoquera l'appel** de cette fonction.

Réponse. 3,5 pts

Cf les TP de la semaine 4 pour utilisation du systick.

Simplement transcrire les phrases ci-dessus en instructions, utiliser les variables et macros données en début d'énoncé, utiliser la fonction *allume_leds* de la question 3 pour "mettre le feu" (allumer la led) au vert ou au rouge.

Q6. On considère que le feu sur la route principale est initialement au vert.
Expliquer comment compléter le code de la fonction *main* ci-dessous, et **écrire le code C** correspondant.

```
int main (void) {
  coldstart();
  config_leds_pins();
  systick_config(RELOAD_VAl_350MS);
  init_rng();
  // et diverses autres initialisations (on ne demande pas de compléter)...
  // allumer feu vert au début
  // A COMPLETER
  // rien d'autre à faire dans l'application
  // A COMPLETER
  return 0;
}
```

Réponse. 1 pt

Idem ci-dessus pour "allumer le feu" au vert.

Et cf TP2 semaine 4, en particulier compréhension forme du *main* dans question 6.

Q7. Nous allons enfin donner la possibilité de ***désactiver le composant RNG lorsque le bouton poussoir est appuyé.*** Lorsque le bouton sera relâché, le composant sera réactivé.
Le bouton est supposé configuré afin qu'une interruption soit émise chaque fois qu'il est soit appuyé soit relâché.

Recopier et, en utilisant les définitions fournies, **compléter** le code de la fonction `exti0_isr`, gestionnaire d'interruptions pour le bouton poussoir. **Expliquer** ce code.

```
void exti0_isr() {
  uint32_t vbp = 0;
  // recuperer dans vbp la valeur du bouton poussoir
  // A COMPLETER
  if (  A COMPLETER  ) {   // si le bouton est appuyé
    // arreter le RNG
    // A COMPLETER
  }
  else { // si le bouton est relaché
    // relancer le RNG
    // A COMPLETER
  }
}
```

Réponse. 4 pts

Même principe question 1 et suivantes pour accès à la valeur du bouton.

Et cf bases vues en semaine 1 et TP2 semaine 4, en particulier toutes les questions sur positionnement d'un bit à 1 / positionnement d'un bit à 0

Exercice 2. *6 points*

Nous considérons un processeur fictif doté d'une mémoire de 1 Mo (c'est-à-dire 1024 Ko), organisée en 4 boîtiers.
Les bits représentant le numéro du boîtier seront placés en bits de poids fort dans les adresses.

Tout identique aux questions du TD semaine 2...

Q1. Combien de bits sont nécessaires pour coder les adresses de cette mémoire ? **Expliquer**.

Réponse. 0,5 pt

Q2. Les adresses étant codées sur le nombre de bits exact que vous avez identifié à la question 1., quel est **l'intervalle** des valeurs possibles des adresses **en hexadécimal** ? **Expliquer**.

Réponse. 0,5 pt

Q3. Combien de bits sont nécessaires dans une adresse pour représenter le **numéro** du boîtier, et quels seront ces numéros ? **Expliquer**.

Réponse. 0,5 pt

Q4. Quelle est la **taille** de chaque boîtier, en octets (en hexadécimal) ? **Expliquer** le calcul.

Réponse. 0,5 pt

Q5. Donner l'**intervalle** d'adresses de chacun des boîtiers **en hexadécimal**.

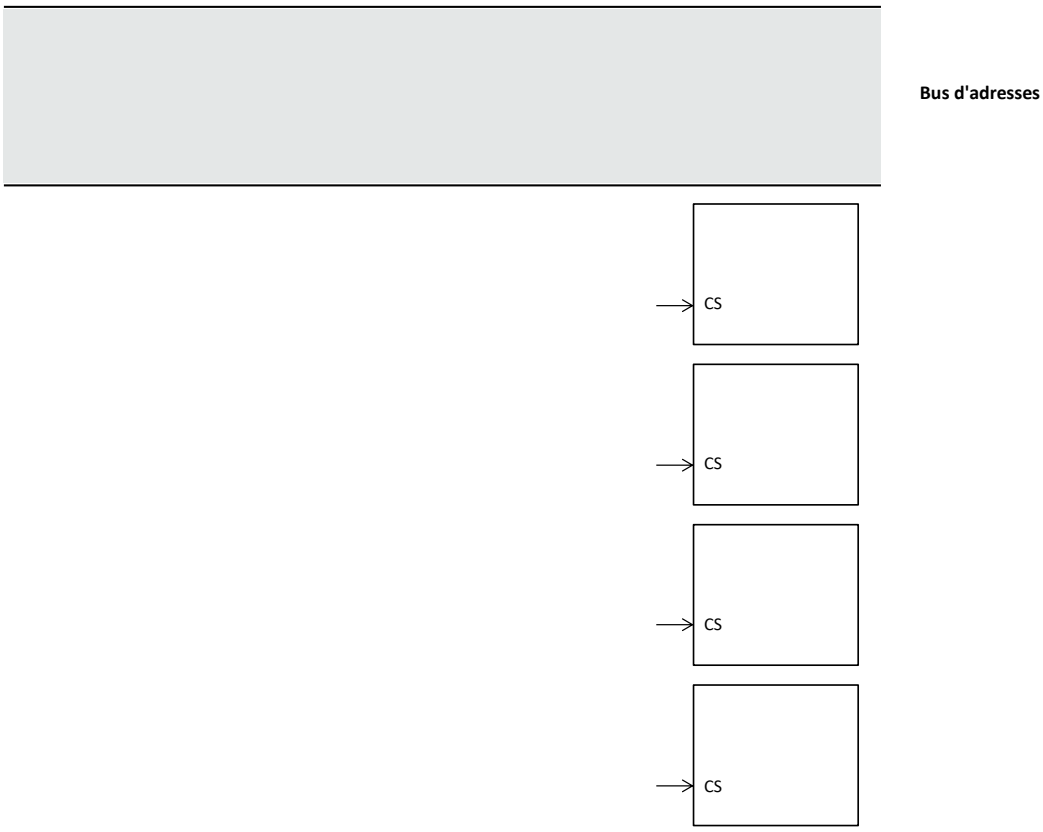
Réponse. 1 pt

Q6. Dans **quel boîtier**, et à **quelle adresse** dans ce boîtier, se trouve la donnée d'adresse A=5020D ? **Expliquer** le calcul.

Réponse. 1 pt

Q7. En conclusion, **expliquer** comment **les connexions matérielles** entre le **bus** d'adresse et les **entrées Adresse et CS** (chip select, à 1 si le boîtier est sélectionné) de chaque boîtier devront être réalisées et compléter le schéma ci-dessous selon ces explications.

Réponse. 2 pts



Annexe. Rappels sur quelques caractéristiques de la carte STM32F4

• Les 4 **leds** sont connectées au port **GPIOD**. Plus précisément : la led verte est connectée au pin 12 de ce port, la led orange est connectée au pin 13, la led rouge est connectée au pin 14,

et la led bleue est connectée au pin 15.

Le **bouton utilisateur** est connecté au port **GPIOA**, pin numéro `PIN_BUTTON` (la valeur 1 pour ce pin signifie bouton appuyé, et la valeur 0 bouton relâché).

Pour cet exercice, les définitions suivantes seront fournies :

```
#define GPIOA_MODER_ADDR 0x40020000
#define GPIOA_IDR_ADDR 0x40020010
#define GPIOA_ODR_ADDR 0x40020014

#define GPIOD_MODER_ADDR 0x40020C00
#define GPIOD_IDR_ADDR 0x40020C10
#define GPIOD_ODR_ADDR 0x40020C14
```

• Le **systick** est un périphérique capable de déclencher une interruption périodiquement, avec un intervalle de temps choisi. Pour ce faire, il utilise un décompteur.
Le nom du gestionnaire d'interruptions associé au systick est `sys_tick_handler`.

• Le **Random Number Generator** (**RNG**) a pour fonction de générer des nombres aléatoires.

Pour qu'il soit opérationnel en mode interruption, il faut :
- activer l'interruption en positionnant à 1 le bit IE du registre RNG_CR.
- activer la génération de nombres aléatoires en positionnant à 1 le bit RNGEN du registre RNG_CR.
Et le nom du gestionnaire d'interruptions correspondant est `hash_rng_isr`.

Lorsqu'un nouveau nombre aléatoire sera supposé disponible, nous nous en assurerons en vérifiant que le bit DRDY du registre RNG_SR est à 1 (ce bit est automatiquement remis à 0 à la lecture de la donnée).

NOTE. Nous avons vu en TP que ce composant peut être sujet à deux types d'erreur en cours de fonctionnement. Pour simplifier, nous ne considérerons pas la gestion de ces erreurs ici.

Le fichier header pour le RNG de la bibilothèque *libopencm3* est rappelé ci-dessous.

```
/* --- Random number generator registers ------------------------------- */

/* Control register (NB. Contenu du reg) */
#define RNG_CR                  MMIO32(RNG_BASE + 0x00)

/* Status register (NB. Contenu du reg) */
#define RNG_SR                  MMIO32(RNG_BASE + 0x04)

/* Data register (NB. Contenu du reg) */
#define RNG_DR                  MMIO32(RNG_BASE + 0x08)

/* --- RNG_CR values --------------------------------------------------- */

/* RNG ENABLE */
#define RNG_CR_RNGEN            (1 << 2)

/* RNG interrupt enable */
#define RNG_CR_IE               (1 << 3)

/* --- RNG_SR values --------------------------------------------------- */

/* Data ready */
#define RNG_SR_DRDY       (1 << 0)

/* Clock error current status */
#define RNG_SR_CECS       (1 << 1)

/* Seed error current status */
#define RNG_SR_SECS       (1 << 2)

/* Clock error interrupt status */
#define RNG_SR_CEIS       (1 << 5)

/* Seed error interrupt status */
#define RNG_SR_SEIS       (1 << 6)
```